FaltaUno: Software Engineering and Architecture Documentation

Institution: RIWI Academy
Integrated Project: FaltaUno
Methodology: Scrum / Agile Development
Base Architecture: Monorepo (SPA Frontend, REST API Backend, SQL DB in 3NF)
Repository link: https://github.com/JayzirTech/FaltaUno-CasptoneProject.git

---

1. Project Name

FaltaUno

2. Technological Justification

The selection of the technological stack for FaltaUno obeys strict criteria of performance, maintainability, and transactional efficiency:

- **Vanilla JavaScript (ES6+) for the Frontend:** By dispensing with heavy frameworks (such as React or Angular), third-party dependency overhead and complex build processes are eliminated. This ensures that the Single Page Application (SPA) remains lightweight, loads instantly on mobile devices, and manipulates the DOM directly with optimal performance.
- **Node.js and Express for the Backend:** Employing JavaScript on both the client and the server unifies the language of the Monorepo, accelerating the development speed. Express offers a minimalist environment ideal for building a fast REST API, with highly efficient asynchronous request handling for transactional flows.
- **Relational SQL Database in Third Normal Form (3NF):** The nature of the business requires absolute consistency. A robust relational model allows enforcing integrity constraints, atomic transactions, and strict concurrency control through foreign keys. This is indispensable to prevent critical errors such as the over-booking of slots in a sports match.

3. Identified Problem

In amateur football, the sudden cancellation of one or more players just a few hours before a match begins is a recurring issue that causes friction and inefficiency. Organizers rely on closed and rudimentary communication channels (such as informal WhatsApp groups or direct phone calls), which have limited reach. When a participant drops out at the last minute, the inability to find a replacement quickly results in the total cancellation of the match. This leads to financial losses due to non-refundable field rental deposits, frustration among players who did attend, and dead hours in the infrastructure of sports centers.

4. General Objective

To design and develop an interactive, centralized web platform under the Single Page Application (SPA) architecture that allows amateur football match organizers to publish available spots and interested players to apply in real-time, streamlining sports matchmaking without page reloads and ensuring data integrity through a secure transactional backend.

5. Specific Objectives

- Implement a reactive and fluid user interface (SPA) using exclusively HTML5, CSS3, and Vanilla JavaScript, handling internal navigation through a client-side router.
- Structure a REST API with Node.js and Express that exposes secure and documented endpoints for the complete lifecycle of matches and applications.
- Design and normalize the relational database schema applying the Third Normal Form (3NF) to eliminate redundancies and ensure referential integrity.
- Develop a centralized authentication system using JSON Web Tokens (JWT) and credential encryption with the Bcrypt hash algorithm to protect community data.
- Guarantee concurrency control in the database to accurately update and deduct available slots in real-time when the organizer approves an applicant.

6. Defined MVP (Minimum Viable Product)

The FaltaUno MVP focuses strictly on solving the core sports matchmaking flow, limiting the scope to the following essential features:

MVP Module	**Included Functionalities**
User Authentication	Registration and login forms. Generation of JWT to protect private routes in the SPA.
Match Publishing	Creation of matches by organizers indicating date, time, location, required slots, and price per person.
Public Feed and Filters	Interactive wall rendering open match cards, with dynamic filters in the DOM by date and price.
Matchmaking Module	Asynchronous action for a player to apply to a match and track the status (Pending, Accepted, Rejected).
Organizer Dashboard	Internal view for the match creator to audit incoming requests and manually manage player admission, dynamically updating the available slots.

Explicit exclusions from the MVP: Integrated online payment gateways (money is handled via cash or external transfer), internal chat systems, user profiles with reputation/star ratings, and support for multiple sports.

7. Project Scope

The scope encompasses frontend design, REST API development on the backend, database

normalization in 3NF, unit and integration testing of core endpoints, and deployment of the monorepo environment in an integrated local architecture. Project management is limited to three sprints defined by the agile methodology of the 6-developer team, concluding with the delivery and defense of the functional MVP.

8. User Stories (Product Backlog)

Structured under the formal Gherkin standard (Given-When-Then):

ID & Title	Functional Description	Acceptance Criteria (Gherkin)
F1-US01: Account Registration	As a new football player, I want to register by entering my personal data, In order to have a profile to interact with the system.	**Given** the user is in the registration view. **When** they enter a duplicate email or an invalid format. **Then** the system blocks submission and displays a red alert without refreshing the browser. **Given** all mandatory fields are correct. **When** they submit the form. **Then** the backend encrypts the password with Bcrypt, stores it, and redirects to the login.
F1-US02: Secure Login	As a registered user, I want to log in by introducing my credentials, In order to securely access private features.	**Given** the user is in the login view. **When** they send an incorrect password. **Then** the backend denies access sending an HTTP 401 status code. **Given** the credentials match the records. **When** they click "Login". **Then** a JWT is generated, the frontend stores it in localStorage, and grants access to the private feed.
F1-US03: Match Publishing	As a match organizer, I want to register a new match with its specifications, In order to externally summon the missing players.	**Given** the organizer is logged in and loads the creation form. **When** they try to enter a past date. **Then** client-side validation stops the request

ID & Title	Functional Description	Acceptance Criteria (Gherkin)
		immediately. **Given** the date, time, price, and slot data are valid. **When** they confirm creation. **Then** the record is created associated with their ID and the match appears immediately on the public feed.
F1-US04: Feed with Filters	As an applicant player, I want to see the feed of available matches and apply DOM filters, In order to find a compatible game quickly.	**Given** the user loads the main feed. **When** the view finishes mounting in the SPA. **Then** an asynchronous fetch request is executed to dynamically inject match cards into the DOM. **Given** the user alters the price or date filter inputs. **When** the input changes state. **Then** the DOM re-renders isolating only the cards that meet the criteria without reloading the page.
F1-US05: Asynchronous Application	As a free player, I want to apply to a match with vacancies, In order to request a slot in the lineup from the creator.	**Given** the player views a match card with available slots. **When** they click the "Apply" button. **Then** a POST request is sent to the server, the application is inserted as 'Pending', and the button changes visually to 'Applied'.
F1-US06: Organizer Management	As the match organizer, I want to accept or reject applications from my dashboard, In order to selectively control who joins the match.	**Given** the creator audits candidates in their administration view. **When** they click "Accept". **Then** the application status changes to 'Accepted' and the available slots in the matches

ID & Title	Functional Description	Acceptance Criteria (Gherkin)
		table automatically decrease by 1. **Given** available slots drop to 0. **When** the last acceptance is processed. **Then** the general status of that match automatically changes to 'Full'.

9. Solution Architecture

The system adopts a decoupled and clean architectural approach, organized through a Monorepo that strictly separates responsibilities:

- **Client Layer (Frontend SPA):** A native Single Page Application built with HTML5, CSS3 (SASS for modular styling), and Vanilla JavaScript. A JS-based router intercepts routes and dynamically rewrites the main container's content without page reloads, interacting with the backend via asynchronous fetch() calls.
- **Server Layer (Backend API REST):** A server developed on Node.js using Express. It implements a clear separation through layers of Routes, Controllers, and Middlewares. Middlewares intercept requests to validate data schemas and decode the session JWT before allowing access to controllers.
- **Persistence Layer (Data):** Relational SQL database, connected to the backend through an optimized connection pool, securely isolating transactional logic.

Project Structure (Monorepo)

```
├── backend
│   ├── controllers
│   │   ├── applications.controller.js
│   │   ├── auth.controller.js  \
│   │   └── matches.controller.js
│   ├── database
│   │   ├── db_config.js
│   │   ├── schema.sql
│   │   └── seeders.sql
│   ├── middleware
│   │   ├── auth.middleware.js
│   │   └── validator.middleware.js
│   ├── routes
│   │   ├── applications.routes.js
│   │   ├── auth.routes.js
│   │   └── matches.routes.js
│   ├── settings
│   │   └── environment.js
```

```
│   ├── server.js
│   └── README.md
├── frontend
│   └── README.md
├── .gitignore
└── README.md
```

10. Data Model (Third Normal Form - 3NF)

The relational design mitigates redundancies and ensures referential integrity by strictly adhering to 3NF rules:

Table: users

Stores identity data and secure credentials for players and organizers.

Field	Data Type	Constraints	Description
id	INT / UUID	PK, Auto-increment	Unique identifier of the user.
full_name	VARCHAR(100)	NOT NULL	Real name of the player.
email	VARCHAR(100)	NOT NULL, UNIQUE	Unique email address used for login.
password	VARCHAR(255)	NOT NULL	Security hash generated using Bcrypt.
phone	VARCHAR(20)	NOT NULL	Phone number for fast coordination between users.
preferred_position	VARCHAR(50)	NULL	Optional field for the player's position on the field (e.g., Forward, Goalkeeper).

Table: matches

Represents the match invitations created on the platform.

Field	Data Type	Constraints	Description
id	INT / UUID	PK, Auto-increment	Unique identifier of the match.

Field	Data Type	Constraints	Description
creator_id	INT / UUID	FK (users.id), NOT NULL	Relationship with the organizing user.
date	DATE	NOT NULL	Date of the match.
time	TIME	NOT NULL	Exact start time.
location	VARCHAR(150)	NOT NULL	Field name or physical address.
total_slots	INT	NOT NULL	Maximum number of field players required.
available_slots	INT	NOT NULL	Number of vacant slots (updated in real-time).
price	DECIMAL(10,2)	NOT NULL, DEFAULT 0.00	Individual fee to reserve the field.
status	VARCHAR(20)	NOT NULL, DEFAULT 'Open'	Match phase ('Open', 'Full', 'Cancelled').

Table: applications

Breaks down the many-to-many relationship between users and matches to strictly conform to 3NF.

Field	Data Type	Constraints	Description
id	INT / UUID	PK, Auto-increment	Identifier of the application.
match_id	INT / UUID	FK (matches.id), NOT NULL	Relationship with the match being applied to.
player_id	INT / UUID	FK (users.id), NOT NULL	Relationship with the applying player.
status	VARCHAR(20)	NOT NULL, DEFAULT 'Pending'	Application status ('Pending', 'Accepted', 'Rejected').